Qualcomm Technologies, Inc.

SW6100 Boot and CoreBSP Architecture

User Guide

80-74841-82 Rev. AB

October 21, 2025

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
hyungchul.won@lge.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AB	October 2025	Editorial changes were made to the doc. Must be read in its entirety.
AA	June 2025	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
hyungchul.won@lge.com

Contents

1 Overview 7

2 BOOT Architecture 8

2.1 Primary boot loader (PBL) 8
2.2 eXtensible Boot Loader (XBL) 8
2.3 Unified extensible firmware interface (UEFI) 9
2.4 ABL (APSS Boot Loader) 9
2.5 HLOS 9

3 Cold boot architecture 10

4 Boot software migration details 12

4.1 Binary image differences 12
4.2 Understanding and comparing BOOT.XF and BOOT.MXF folder structures for boot configuration 13
4.3 XBL system controller (SC) 13
4.4 XBLConfig binary overview 14
4.5 XBL devicetree configuration for XBL SC 14
4.6 XBL devicetree interfaces 15
4.7 Qualcomm devicetree editor (QDTE) 19
4.8 Debug methods 24
4.8.1 PBL debug 24
4.8.2 PBL/XBL log 24
4.8.3 UEFI Log 24
4.8.4 ABL Log 25
4.8.5 HLOS Log 25
4.8.6 Force EDL mode 25
4.8.7 Boot options using fuses and GPIOs 26
4.9 System debug image (SDI) 26
4.9.1 Reset debug design 26
4.9.2 Reset debug flow (first pass) 27
4.9.3 Reset Debug Flow (Second Pass) 28

5 Unified Extensible Firmware Interface(UEFI) 30

5.1 UEFI driver 30
5.2 UEFI application 30
5.3 UEFI library 31

Qualcomm Confidential – May Contain Trade Secrets
2026-02-27 06:37:54 GMT
hyungchul.won@lge.com

A References 39
A.1 Related documents 39
A.2 Acronyms and terms 39

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
hyungchul.won@lge.com

Figures

Figure 4-1 Figure: Settings for Windows signature configuration 20
Figure 4-2 Seetings for Windows signature configuration 21

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
hyungchul.won@lge.com

Tables

Table 4-1 Configuration for Linux/Windows 19
Table 4-2 Checking ABL log 25
Table 5-1 File definitions (UEFI driver and application INF files) 36

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
hyungchul.won@lge.com

1 Overview

This guide outlines the boot features supported by the SW6100, explains the architecture of a cold boot (initiated from a power-off state), and describes the available interfaces.

It also provides migration guidance from the SW5100 boot loader, along with instructions for developing Unified Extensible Firmware Interface (UEFI) applications, configuring devicetree (DT) interfaces, referencing boot-up logs, and using supported debugging methods.

Qualcomm
Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
hyungchul.won@lge.com

2 BOOT Architecture

The boot architecture uses boot loaders and board support package (BSP) images at each stage of boot.

2.1 Primary boot loader (PBL)

- Establishes a secure root-of-trust (RoT) and secure boot process for applications.
- Identifies the primary storage device and loads the secondary boot loader called the eXtensible Boot Loader (XBL).
- If the system encounters any recoverable error while loading the XBL image, the PBL enters the emergency download (EDL) mode. For more information about how to detect EDL mode, see Enumeration of EDL device manager.
- Loads XBL segments into the boot SoC internal memory. For more information about how PBL loads the XBL segments, see Cold boot architecture.

2.2 eXtensible Boot Loader (XBL)

- Initializes the hardware, firmware images, CPU cache, MMU, boot device, PMIC, and DDR.
- Sets up the RAM dump USB driver, USB charging, thermal check, power management integrated circuit (PMIC), and low-power double data rate (LPDDR) clock functions.
- Collects the RAM dump over USB onto the host PC.
- Loads and verifies the Qualcomm® Trusted Execution Environment (Qualcomm TEE), Qualcomm Hypervisor, and UEFI image.

Qualcomm Confidential – May Contain Trade Secrets 2020-02-27 09:27:54 GMT hyungsul.kim@lge.com

2.3 Unified extensible firmware interface (UEFI)

UEFI is the software interface between an operating system (OS) and the platform firmware.

It includes data tables with platform information, along with the boot and runtime service calls that the OS and its loader can use. Together,they create a standard environment for booting an operating system and running UEFI applications.

Qualcomm uses Tianocore EDK2, an open-source implementation available at http://www.tianocore.org/edk2/, which follows the UEFI specification.

The system offers two build options, which are:

- Retail build: Has minimal debug features and an optimal memory footprint, making it ideal for production environments.
- Debug build: Ideal for development environments, with all debug features enabled.

For developing a UEFI application, see Unified Extensible Firmware Interface(UEFI) chapter.

2.4 ABL (APSS Boot Loader)

APSS boot loader is the Linux/Android boot loader which performs:

- Loads and executes the Linux/Android kernel image.
- Supports fastboot protocol that supports flashing a partition, erasing a partition, changing the state of the device, etc.

2.5 HLOS

HLOS stands for High-Level Operating System.

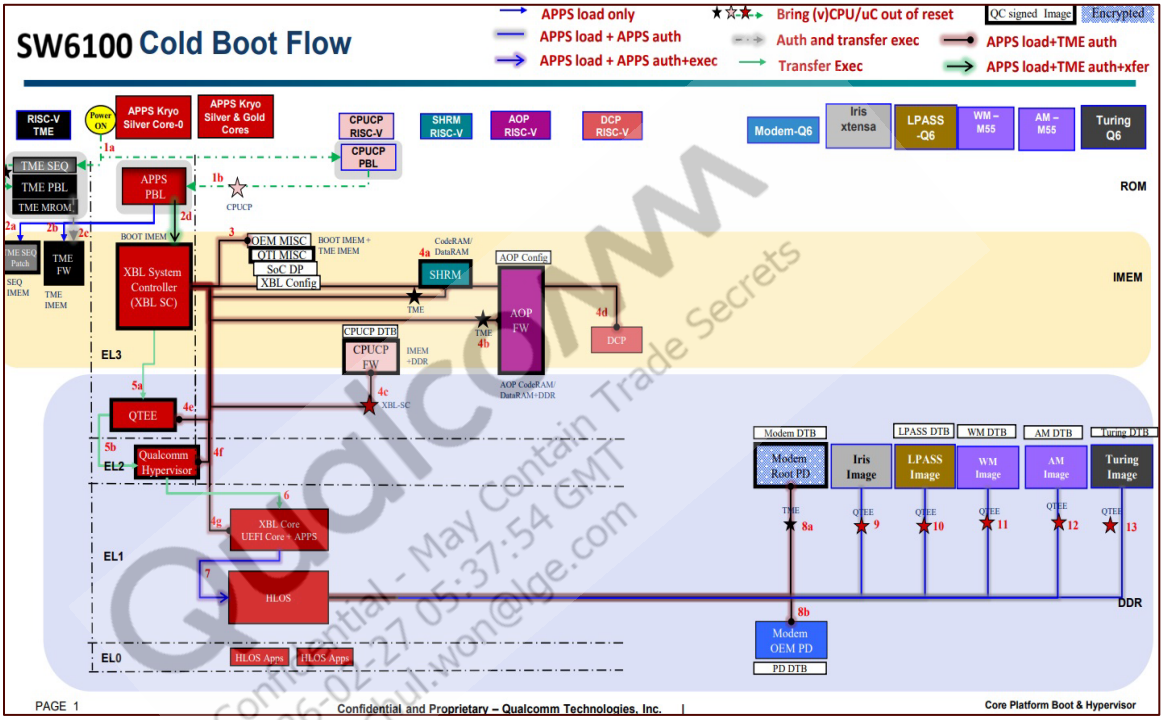
- It refers to the Linux-based operating system that runs on the Application Processor Subsystem(ASSS).
- It is responsible for managing the application processor and providing a platform for running user applications and high-level system software.
- It manages system resources, executes user applications, and interacts with other subsystems to provide a robust and efficient computing environment.

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:27:54 GMT hyungchul.yoon@lge.com

3 Cold boot architecture

A cold boot starts the system from the power-off state. This process begins with the PBL.

The following figure shows the cold boot flow.



The cold boot sequence involves the following steps:

1a-1b. After the chipset is reset, both TME core and CPUCP core come out of reset. CPUCP brings Applications Qualcomm Custom cores out of reset and executes TME ROM firmware and application primary boot loader (APPS PBL).

On Applications Qualcomm Custom core, APPS PBL initializes hardware (timer, PRNG, clocks, and so on), enables caches and MMU, and detects the boot device according to the boot option configuration:

□ Default boot option: eMMC > EDL mode

2a. APPS PBL loads TME firmware from the boot device to TME IMEM.

2b. TME ROM firmware authenticates TME RAM firmware and transfers the execution to TME RAM firmware. TME works as the primary root of trust (RoT) on SoC and initializes early SoC AC policy.

2c. APPS PBL loads XBL SC image from the boot device to boot IMEM and TME ROM authenticates XBL SC. After TME is ready and XBL SC image authentication is passed, TME permits APPS PBL exit to run XBL SC.

3. XBL SC runs in the EL3 mode and initializes memory subsystem (buses, DDR, clocks, and CDT), sets XBL SC AC policy, loads SoC DP, Qualcomm multi-image signature and integrity check (MISC), OEM MISC, load XBL CFG, Qualcomm® Trusted Execution Environment (TEE) Qualcomm® Hypervisor Execution Environment (HEE), AOP CFG, AOP firmware, SHRM, XBL core images (UEFI), executes DDR training, if necessary, saves memory dump using Sahara or to SD card if DLOAD cookie is set, supports watchdog debug retention, USB charging, thermal check, and supports PMIC driver. TME authenticates all images. XBL CFG includes CPR, DDR, and XBL devicetree for other driver settings abstraction.

4a. XBL SC loads the SHRM image from the boot device and TME authenticates and brings it out of reset.

4b. XBL SC loads AOP CFG and AOP image from the boot device and TME authenticates and brings AOP out of reset.

4c. XBL SC loads CPUCP DTB and CPUCP firmware image from the boot device and TME authenticates it. XBL SC brings it out of reset.

4d. XBL SC loads DCP firmware image from the boot device and TME authenticates it.

4e. XBL SC loads Qualcomm TEE image from the boot device and TME authenticates it.

4f. XBL SC loads Qualcomm HEE image from the boot device and TME authenticates it.

4g. XBL SC loads XBL core image from the boot device and TME authenticates it.

5a. XBL SC sets XBL SC exit and Qualcomm TEE boot milestone state, notify to TME then jump to the Qualcomm TEE cold boot.

5b. Qualcomm TEE sets up the runtime secure environment, configures xPU, and supports fuse driver. Qualcomm TEE transfers the execution to Qualcomm HEE and Qualcomm HEE runs at EL2 mode. Qualcomm HEE configures SMMUs and provides virtualization support.

6. Qualcomm HEE transfers the execution to XBL core, which is known as the unified extensible firmware interface (UEFI). UEFI runs at nonsecure EL1 mode. UEFI mounts and runs application boot loader firmware volume (ABL FV), which has fastboot and Linux loader applications.

7. Linux loader application (part of ABL FV) loads and authenticates the HLOS kernel with Verified Boot, jumps to HLOS. HLOS runs at nonsecure EL1 mode.

8a-9b. *HLOS PIL driver loads Modem Root PD and OEM PD images and TME authenticates and brings it out of reset.

9-13. HLOS PIL driver loads all other subsystems to DDR and configures the clocks and power rails, if necessary. HLOS PIL driver executes an SCM call to Qualcomm TEE to request a secure PIL driver, authenticates images, and Qualcomm HEE brings each subsystem out of reset.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 07:37:54 GMT hyunghul.woo@qualcomm.com

4 Boot software migration details

The below table explains the BOOT software migration details from SW5100 to SW6100.

Change	Description	Migration	Impact
eXtensible Boot Loader (XBL)	Qualcomm will not release the XBL loader source code. Instead, it will release a signed XBL loader binary called **XBL SC**. Releasing firehose ddr programmer and RAM dump source code.	XBL common customizations will be supported through XBL devicetree, remaining customizations should be moved into UEFI. Refer devicetree setting guide in XBL Customization Using Device Tree Hardware/Software Settings (80-PE663-2) for more information	High
Firehose programmer	Release firehose ddr programmer image.	You can use Sahara protocol to collect PBL dump. Provide Sahara CMD6 to get PBL error debug information. See Sahara Protocol User Guide (80-N1008-1) for more information.	Medium
Partition table	Separate partition for UEFI image to accommodate XBL-SC change	You should update your own download tools and flash scripts to accommodate the change.	Low
Partition table	Separate partition for XBL RAM dump image to accommodate XBL-SC change	You should update your own download tools and flash scripts to accommodate the change.	Low
System Debug Image (SDI)	A small portion of SDI logic for DDR driver is in XBL region #2, and the remaining is in the Qualcomm TEE image.	SDI function has been distributed to TME, AOP, CPUCP, and TZ. See System Debug Image section for more details	Low

Qualcomm Confidential – May Contain Trade Secrets
2024-05-31 05:37:54 GMT
hyungseok.kim@lge.com

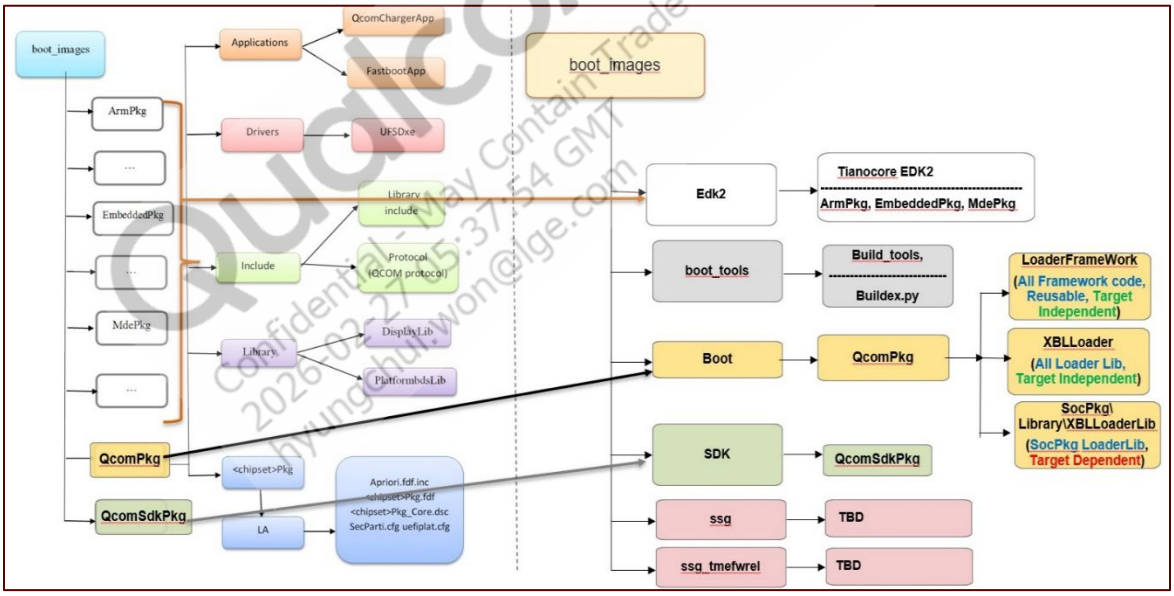
4.1 Binary image differences

SW5100	SW6100	Description
`xbl.elf`	`xbl_s.melf`	`eXtensible Boot Loader is shipped as Binary image`
	`uefi.elf`	`UEFI software image`
	`shrm.elf`	`SHRM image for DDR.`
	`XblRamdump.elf`	`XBL Ramdump software image.`

SW5100	SW6100	Description
xbl_config.elf	xbl_config.elf	XBL config image.
	xbl_config_devprg.elf	Device Programmer image for XBL config image.
	xbl_config_devprgl.elf	
prog_firehose_ddr.elf	xbl_s_devprg_ns.melf	Device Programmer image for programming the images.
prog_firehose_lite.elf	xbl_s_devprgl_ns.melf	
JtagProgrammer.elf	JtagProgrammer.elf	JTAG programmer image used with T32 software to flash the images.
imagefv.elf	imagefv.elf	imageFV binary image
ddi.elf	NA	-

4.2 Understanding and comparing BOOT.XF and BOOT.MXF folder structures for boot configuration

The image below shows the folder layout differences between SW5100 and SW6100.



4.3 XBL system controller (SC)

XBL SC is introduced as part of the XBL 3.0 architecture. The XBL SC image operates in secure EL3 mode and performs secure system initialization. This secure component is referred to as **XBL SC**.

- XBL SC image is released as a Qualcomm-signed binary, without releasing the source code.
  - Qualcomm signed images: `xbl_sc.elf , shrm.elf`

- Qualcomm continues to release the source code to compile XBL CFG, UEFI, DDR device programmer, and XBL RAM dump images.
  - OEM signed images
    - JtagProgrammer.elf
    - uefi.elf
    - xbl_config.elf
    - prog_firehose_ddr.elf
    - prog_firehose_lite.elf
    - XBLRamdump.elf

  XBL SC loads images from the boot device and invokes TME for authentication.

  Separate partitions are allocated for the UEFI image and the XBL RAM dump image.

  Predefined features and configurations are supported through the XBL devicetree configuration.

  Customization should be implemented either within the UEFI or by leveraging the XBL devicetree configuration.

4.4 XBLConfig binary overview

- The XBLConfig binary item packages CPR settings, DDR configurations, and XBL devicetree configurations.
- It's introduced to separate configuration data from the XBL code sequence, enabling modular and flexible management.
- A single configuration file offers:
  - Memory optimizations: CPR, DDR, DTC, and other drivers can split settings into multiple config binary items, allowing selective loading based on device requirements.
  - Flexibility and better management: Config binary items can be tailored according to hardware block versions, chipsets, and other platform-specific parameters.
  - Standalone tooling: Changed config binary items can be converted into ELF binary images using standalone tools, without requiring full compilation or build invocation.
- You can also include their own configurations within the XBLConfig binary item. See the XBL Configuration Guide (80-PE663-1) for more details.
- During testing, QDUTT allows you to override DDR settings through a GUI. For more information, see the QDUTT eCDT User Guide (80-NR058-9).

4.5 XBL devicetree configuration for XBL SC

Qualcomm introduces XBL devicetree configurations to support changes introduced with XBL SC.

You can use these configurations to customize common features through the devicetree. Source files for the Aspen chipset at:
`\boot_images\boot\Settings\Soc\Aspen\`

Qualcomm Confidential – May Contain Trade Secrets 2025-02-25 05:31:54 GMT

XBL DTBs include the following files:

- `pre-ddr-aspen-1.0.dtb`: Used during pre-DDR initialization.
- `post-ddr-aspen-1.0.dtb`: Used during post-DDR initialization.

Qualcomm supports multiple post-DDR DTBs, which you can generate. These DTBs are selected based on the best match to the hardware configuration.

NOTE: Multiple pre-DDR DTBs aren't supported.

For guidance on generating DTB files, see Non-HLOS Customization Using Device Tree Hardware/Software Settings User Guide (80-29623-65).

4.6 XBL devicetree interfaces

Devicetree (DT) nodes define access to boot interfaces and their associated properties. The DT provides a structured representation of the system's hardware information, which is read by the image execution environment—including firmware components such as XBL and UEFI.

You can modify DT properties related to boot to customize various device software features.

For guidance on modifying interfaces and updating DT properties in a prebuilt Devicetree Blob (DTB), see the QDTE tool documentation.

Platform-info

The Platform-info DT provides information about CDT which contains platform information version, platform type, major hardware version, minor hardware version, subtype and OEM-ID.

The platform Info DT file path : `boot_images/boot/Settings/Soc/<chipset>/Core/SocInfra/PlatformInfo/PlatformInfo.dtsi.`

- PlatformInfo version - Specifies the PlatformInfo version of the driver. The supported version number is 0x5.
- Platform type - Specifies the type of platform which is IOT.
- Major hardware version - Specifies the major version information of the platform.
- Minor hardware version - Specifies the minor version information of the platform.
- Subtype - Specifies the subtype for the platform type.
- OEM variant ID - Specifies user defined ID. It's reserved and unused by Qualcomm.
- 0x00 - Qualcomm reference platform.
- 0x01-0xff - OEM variant ID
- Number of key-value pair (KVP)- Specifies dependent key values associated with platform type.

NOTE: OEM variant ID is supported by platform Info version 5.

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyungchul.kim@lge.com

QDTE reference output:

/soc/PlatformInfo	Node		
/soc/PlatformInfo/compatible	Property	STRINGS	'qcom,platforminfo'
/soc/PlatformInfo/PlatformInfo	Property	BYTES	5 8 1 0 0 0 0

Boot devicetree

The boot devicetree properties provide information on how to enable and disable user-initiated emergency download (EDL) in XBL. The boot DT properties also describe the EDL timeout and the USB cable options that can force the system into the EDL mode. The boot-related functionality configures using the boot DT properties.

The boot DT file path is `/boot_images/boot/Settings/Soc/<chipset>/Core/Boot/sw_boot.dtsi`

```
&sw {
        boot {

               compatible = "qcom,boot";

              /*        client trace  verbose */
              config = <0x0000 0x0000 0x0000>;

              /* 0x0 = PCIe_Flashless (default), 0x1 = USB_Flashless */
              force_dump_to_usb = <0x0>;

              /* 0x0 = DISABLE (default), 0x1 = ENABLE default test_mode
behavior, 0x2 = ENABLE one time test */
              test_mode = <0x0>;

              /* DISABLE = 0x0, ENABLE = 0x1(default) */
              xbl_sc_wdog = <ENABLE>;

              /* DISABLE = 0x0, ENABLE = 0x1(default) */
              xbl_ramdump_wdog = <DISABLE>;

              /* Do not skip DDR init = 0x0(default), skip DDR init = 0x1
*/
              #ifdef DEVICE_PROGRAMMER_LITE
              skip_ddr_init_in_EDL = <ENABLE>;
              #else
              skip_ddr_init_in_EDL = <DISABLE>;
              #endif


              /*        default action: vibration is enabled at start of
XBL and disabled before exit */
              /*        DISABLE = 0X0 - turn off vibration, ENABLE = 0x1 -
turn on vibration */
              vibration = <ENABLE>;
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyungchunwong@qc.com

```
            /* save boot logs to storage for each boot in the boot log
partition */
            /* 0x0 = DISABLE (default), 0x1 = ENABLE Log buffer flush,
0x2 = ENABLE streaming logs */
            save_logs_to_media = <0x0>;

            /* Order of the properties in this node must not be changed
as it will break the feature */
            forced_download{
                              /*         Enable(default)/Disable User
initiated EDL in XBL */
                              feature = <ENABLE>;


                              /*                default set to checking
dp low initially and then high withing configured timeout */
                              /*                valid values for this
property are defined in boot_sw_defines.h */
                              check_usb_option =
<CHECK_USB_D_PLUS_HIGH_WITH_TIMEOUT>;


                              /*                 Timeout in ms (maximum
value is 10000 (10 seconds)) */
                              /*                 *Effective only if
checks_dp_high is enabled */
                              check_dp_timeout = <10000>;

                              /*              Option to ignore power key
pressed with VOL+ and VOL- */
                              /*              ENABLE: PWR key press will
be ignored to enter EDL, DISABLE(default): PWR key press + VOL+/VOL- will
not enter EDL*/
                              ignore_pwr_key = <DISABLE>;
                            };


            /* Order of the properties in this node must not be changed
as it will break the feature */
            fedl_timeout{
                         /*     Property version */
                         ver = <BOOT_FEDL_TIMEOUT_PROP_VERSION>;

                         /*        default action: hard reset */
                         /*        valid values for this property are
defined in boot_sw_defines.h */
                         action = <BOOT_FEDL_TIMEOUT_ACTION_HARD_RESET>;
                        };
        };
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyungchul.woo@lge.com

```
};
```

QDTE reference output:

/sw/boot	Node		
/sw/boot/compatible	Property	STRINGS	'qcom,boot'
/sw/boot/config	Property	WORDS	0 0 0
/sw/boot/force_dump_to_usb	Property	WORDS	0
/sw/boot/test_mode	Property	WORDS	0
/sw/boot/xbl_sc_wdog	Property	WORDS	1
/sw/boot/xbl_ramdump_wdog	Property	WORDS	0
/sw/boot/skip_ddr_init_in_EDL	Property	WORDS	0
/sw/boot/vibration	Property	WORDS	1
/sw/boot/save_logs_to_media	Property	WORDS	0
/sw/boot/forced_download	Node		
/sw/boot/forced_download/feature	Property	WORDS	1
/sw/boot/forced_download/check_usb_option	Property	WORDS	1
/sw/boot/forced_download/check_dp_timeout	Property	WORDS	10000
/sw/boot/forced_download/ignore_pwr_key	Property	WORDS	0
/sw/boot/fedl_timeout	Node		
/sw/boot/fedl_timeout/ver	Property	WORDS	1
/sw/boot/fedl_timeout/action	Property	WORDS	1

PMIC

The power management integrated circuit (PMIC) is a specialized electronic component that manages various power supply requirements within a device.

The PMIC DT file path is `boot_images/boot/Settings/Soc/<Chipset>/Core/PMIC/pm.dtsi, access.dtsi.`

See SW6100 PMIC Software User Guide (80-74841-80) for more details.

TLMM

The hardware controls the multiplexing of general-purpose input/output (GPIO) and alternate functions through the top-level mode multiplexer (TLMM). You can configure the GPIOs during the boot process using the DT properties for TLMM.

The TLMM DT file path is `/boot_images/boot/Settings/Soc/<Chipset>/Core/SocInfra/TLMM/<Chipset>-pinctrl.dtsi.`

See SW6100 GPIO Overview (80-74841-92).

Buses

A bus facilitates data transfer among various system components. The DT properties allow for the configuration of the Qualcomm Universal Peripheral (QUP) v3 serial interface device nodes.

The TLMM DT file path is `boot_images/boot/Settings/Soc/<chipset>/Core/Buses/qup_common/<Chipset>-qupv3.dtsi`

See SW6100 Peripheral (UART, SPI, I^2C, I^3C, USB) Overview (80-74841-87).

Qualcomm Confidential – May Contain Trade Secrets
Verified User 2026-02-27 09:37:54 GMT
hyungshul.woo@gmail.com

4.7 Qualcomm devicetree editor (QDTE)

The QDTE graphical user interface (GUI) tool configures and modifies Devicetree Binary (DTB) files by editing the `xbl_config.elf` file.

You can download QDTE from QSC or QPM and launch the UI using the command ./qdte.

QDTE offers the following features:

- Allows for experimentation with configuration settings in a test environment.
- The tool opens/reads a DTB file directly or parses an `xbl_config.elf` file and extracts the embedded DTB files. In the latter case, when you save the file, the tool regenerates the `xbl_config.elf` file with signing.

NOTE: Don't use the QDTE tool to create the devicetree source/devicetree source inclusion (DTS/DTSI) files that represent the input source.

This section explains how to use QDTE tool to do the following:

- Load, parse, edit and save the DTB file.
- Read and write the DTB file from device or on host machines.
- Set up the configuration on your host device, whether it's Linux or Windows.

Set up the configuration on your host device as follows:

Table 4-1 Configuration for Linux/Windows

Property	File path for Linux/Windows host machine
SecTool path: Installation path to SecToolsv2, a tool used to perform secure operations on software images.	▪ Linux:`SECTOOLS="META_PATH"/common/sectoolsv2/ext/Linux/sectools.` ▪ Windows: `SECTOOLS="META_PATH"\common\sectoolsv2\ext\windows\sectools`. For more information on SecToolsv2 for Windows, see SecTools V2: Metabuild Secure Image User Guide.
Profile file: Used for signing the security profile XML files.	▪ Linux: `"META_PATH"/common/sectoolsv2/<chipset>_security_profile.xml.` ▪ Windows:`"META_PATH" \common\sectoolsv2\<chipset>_security_profile.xml.`
Sign cmd json: Used for signing the JSON file from the QDTE installation directory. The test signing mode is used for working on a nonsecure device. For secure devices, see enable secure boot.	▪ Linux: `/local/mnt/qcom/QDTE/test_signing_mode.json.` ▪ Windows:`C:\Program Files (x86)\Qualcomm\QDTE\test_signing_mode.json.`
Property	File path for Linux/Windows host machine
Devprg file: Device programmer file.	▪ Linux: `"META_PATH"/ /boot_images/boot/QcomPkg/SocPkg/Aspen/Bin/LW/DEBUG or RELEASE/ xbl_s_devprg_ns.melf` ▪ Windows: `"META_PATH"\\boot_images\boot\QcomPkg\SocPkg\Aspen\Bin\LW\DEBUG or RELEASE \ xbl_s_devprg_ns.melf`
Flash type: Type of storage, which in this case is eMMC.	-

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 PM hyungchul.no@qdc.com

Figure: Settings for Linux signature configuration

Figure 4-1 Figure: Settings for Windows signature configuration

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyungchul.won@lge.com

Figure 4-2 Seetings for Windows signature configuration

Use the following procedure to configure the DTB files using QDTE. This procedure is applicable only to XBL and UEFI DTB.

1. To open the `XBLConfig ELF` file, go to *File* > *Open DTB Elf* > *From Build*.

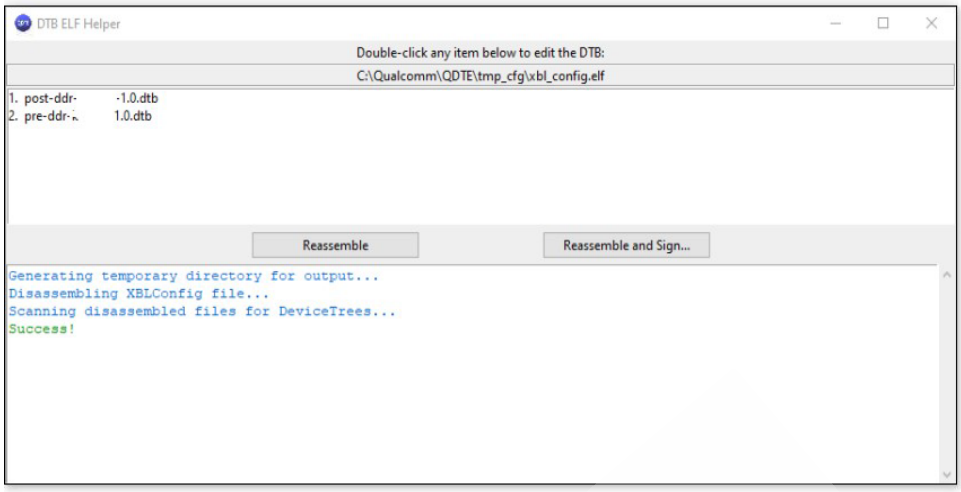
   `"META_PATH"\\boot_images\boot\QcomPkg\SocPkg\Aspen\Bin\LW\DEBUG or RELEASE \xbl_config.elf`

- If the file is invalid or corrupted, the system generates an error. To prevent the error, ensure that you download the valid boot binaries, including `xbl_config.elf`, and use them with the XBL configuration. The binary releases the boot component, which is part of the firmware binary.
- If the file loads successfully, a tree view appears, populated with the DTB elements.

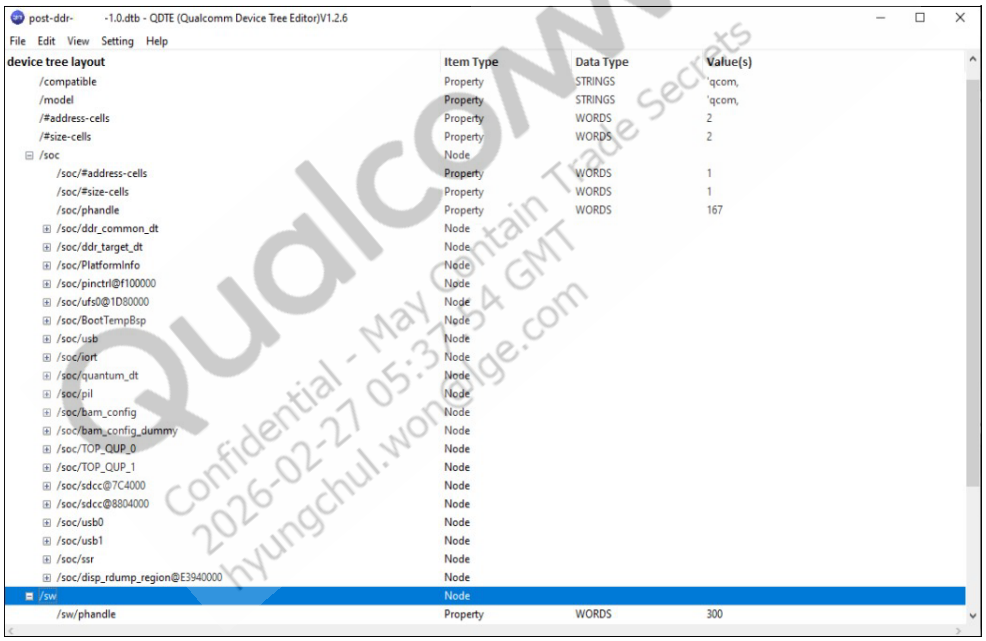
2. Double-click on the `XBLConfig ELF` file to edit it.

The primary QDTE interface displays the file.

The figure shows an XBLConfig helper window displaying the disassembled results of an XBLConfig file containing two DTBs.

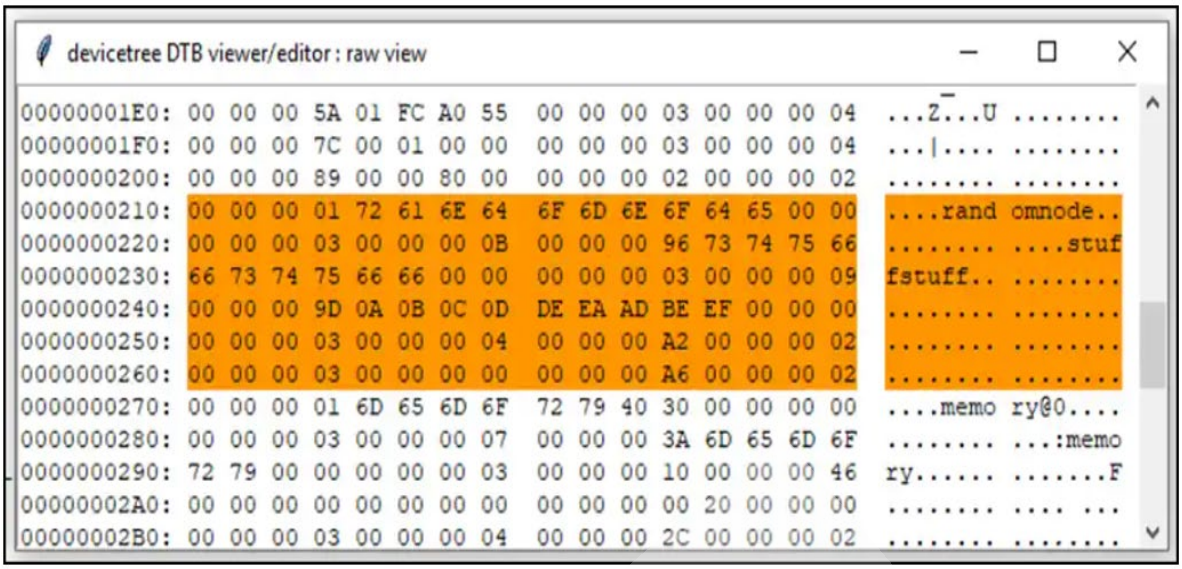


The figure shows an example file opened in a tree view with all four columns populated.



3. To discard all the changes and reload the file from the disk (if the file undergoes changes on the disk while the QDTE tool is open), select ***File > Reload File***.
4. To edit a property, you can either right-click and select **Edit** or double-click to open a dialog box. This dialog box allows you to edit both the property name and value.
5. To view the DTB file as a binary hexadecimal dump, go to **View > Raw**, which displays the exported DTB file.

The following example of the raw view shows the offset, hexadecimal value, and corresponding character columns, with a node highlighted in orange.



6. To remove all highlights, go to **View > Clear Highlights**.
7. To save the modified DTBs, you can either go to File > Save (or press CTRL + S) to overwrite the existing file or go to File > Save Copy As (or press CTRL + SHIFT + S) to save a copy of the DTB to a new file.
8. To generate a summary of the changes, select File > Export Change Report.

   Creates a JSON file listing all changes made to the DT in the session, along with the result of the DTB after the operation.
9. To save the modified DTB files within the main QDTE, go to the XBLConfig helper window, and click Reassemble and Sign. Saves your modified XBLConfig file.
10. Next, choose a name for the output file.
11. The toolchain re-assembles the ELF using XBLConfig. If an error message indicates that a signed XBLConfig wasn't generated, it downgrades the error to a warning and shows a success message instead.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 03:37:54 KST hyungchul.workedge.com

4.8 Debug methods

This section explains about supported debug methods in boot loaders.

4.8.1 PBL debug

PBL Sahara CMD 6 dump: Supports Sahara read debug data CMD 0x6 on SW6100.

Run `QSaharaServer.exe -p \\.\COMxx -k -v 3 -c 6` to collect PBL Sahara CMD 6 dump. The device should be in EDL mode while running this command.

COMxx in the command means EDL (0x9008) port number.

NOTE: Attach the PBL dump to a Salesforce case. The Qualcomm support team parses the error information.

4.8.2 PBL/XBL log

The sample primary boot loader/eXtensible boot loader (PBL/XBL) log indicates the start and end of the PBL and XBL.

The following table lists log entries that can help you verify the start and end of the boot loaders.

Log entry	Description
`S - Boot Interface: eMMC`	Indicates the booting of software images from eMMC as the boot image is stored in eMMC
`S - PBL Patch Ver: 1`	Indicates the start of PBL.
`S - 69043 - PBL, End`	Indicates the end of PBL.
`B - 79574 - SBL1, Start`	Indicates the start of XBL. Both SBL1 and XBL see the same secondary boot loader.
`B - 1394338 - SBL1, End`	Indicates the end of XBL.
`D - 1318210 - SBL1, Delta`	Indicates the time difference in microseconds between start and end of XBL.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:59 GMT hyungchul.won@qce.com

4.8.3 UEFI Log

The UEFI log provides information about the start and end of UEFI.

The table lists the log entries that can help you verify the start and end of UEFI

Table: Checking UEFI log	
Log entry	**Description**
`UEFI Start [ 1568]`	Indicates the start of UEFI
`UEFI Total : 1261 ms`	Indicates the end of UEFI

4.8.4 ABL Log

The ABL log provides information about the start and end of ABL.

The table lists the log entries that can help you verify the start and end of ABL.

Table 4-2 Checking ABL log

Log entry	Description
POST Time [5196] OS Loader	Indicates the start of ABL
[6741] UEFI End	Indicated the end of ABL

4.8.5 HLOS Log

The sample log signifies the start of the Linux boot process.

Use this log entry to verify if the Linux boot process has finished successfully.

```
[ 0.000000][    T0] Booting Linux on physical CPU 0x0000000000 [0x412fd050]
[ 0.000000][    T0] Linux version 6.12.23-android16-4-maybe-dirty-debug
(kleaf@build-host) (Android (12833971, +pgo, +bolt, +lto, +mlgo, based on
r536225) clang version 19.0.1
(https://android.googlesource.com/toolchain/llvm-project
b3a530ec6537146650e42be89f1089e9a3588460), LLD 19.0.1) #1 SMP PREEMPT Thu
Jan  1 00:00:00 UTC 1970
[0.000000][    T0] Machine model: Qualcomm Technologies, Inc. Vienna WDP
```

4.8.6 Force EDL mode

The three options to enter the Force EDL mode are:

1. `FORCE_USB_BOOT (GPIO_79)`: See SW6100 Technical reference manual (80-83951-5) for more details. It can be enabled by pulling up `FORCE_USB_BOOT GPIO` pin, `FORCE_USB_BOOT` is scanned in PBL. The `FORCE_USB_BOOT` forces the device to enter EDL regardless of the state of the `BOOT_CONFIG` GPIOs or `FAST_BOOT_SEL` fuses.
2. Options to enter EDL mode in `XBL_SC`.
   - Vol up + Vol down buttons + 'USB D+ grounded' cable in `XBL_SC`.
   - Vol up + Vol down buttons + Regular USB grounded in `XBL_SC`.
   - Ignore Power Key option is also configurable for the above options to enter EDL in `XBL_SC`.
     This feature can be enabled and configured in XBL devicetree settings. Feature enable/disable, Type of USB cable and ignore power key status are the configurations that can be modified.

     The configuration is read in `XBL_SC` and checked against the status of keys and USB. If keys status + USB conditions match the configuration, the device runs into the Force EDL mode.
3. Vol up + Vol down buttons in UEFI : Detects both Volume up and Volume down buttons pressed, the device runs into the Force EDL mode.

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 06:37:54 GMT hyunchul.lee@qualcomm.com

4.8.7 Boot options using fuses and GPIOs

See SW6100 Technical reference manual (80-83951-5).

BOOT_CONFIG[3..1] (FAST_BOOT)		Boot sequence	Boot option using fuses				Boot option using GPIOs			
			OEM_CONFIG FUSE: FAST_BOOT_SELECT[3:0				BOOT_CONFIG[3..1]			
			Bit 3	Bit 2	Bit 1	Bit 0	GPIO [148]	GPIO [149]	GPIO [87]	GPIO [6]
Boot options	0b000	eMMC EDL (USB)	0	0	0	0	0	0	0	0
	0b001	USB boot only	0	0	0	1	0	0	0	1
	0b010	eMMC EDL (USB)	0	0	1	0	0	0	1	0

4.9 System debug image (SDI)

There is no standalone System Debug Image (SDI); instead, its functionality is distributed across subsystems including TME, AOP, CPUCP, and TZ.

- Cold boot behavior: During cold boot, the SDI components within the TME and AOP-SDI images activate reset debug features based on the defined debug policy.
- Abnormal reset handling: Events such as abnormal resin triggers, watchdog resets, and thermal resets initiate the reset debug sequence.
- Context collection: SDI collaborates with the distributed subsystems to gather system context and flush it to DDR buffers for analysis.

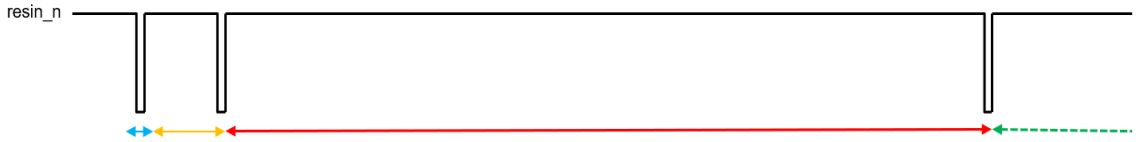
Qualcomm Confidential - May Contain Trade Secrets 2025-02-27 05:37:54 GMT hyungchul.won@lge.com

4.9.1 Reset debug design

1. Configuration:
   - TME and AOP enable a reset debug in cold boot per the debug policy.
   - HLOS populates dump buffers for SDI and distributed subsystems.
2. Wake-up step:
   - After an abnormal reset occurs with the reset debug enabled, resin_n is blocked, and the wake-up step starts.
3. Preparation step:
   - Hardware reacts to prepare itself in a certain state before resetting the device and puts DDR into self-refresh.
4. First pass:
   - TME brings AOP to run AOP-SDI.
   - AOP brings SHRM to take DDR out of self-refresh.
   - AOP triggers key subsystems to collect dumps.
   - AOP dumps LLCC and ETBs

- CPUCP to determine not to expose the call stacks (GPRs) if the crash occurs in the secure space and nonsecure data RAM dump is enabled.

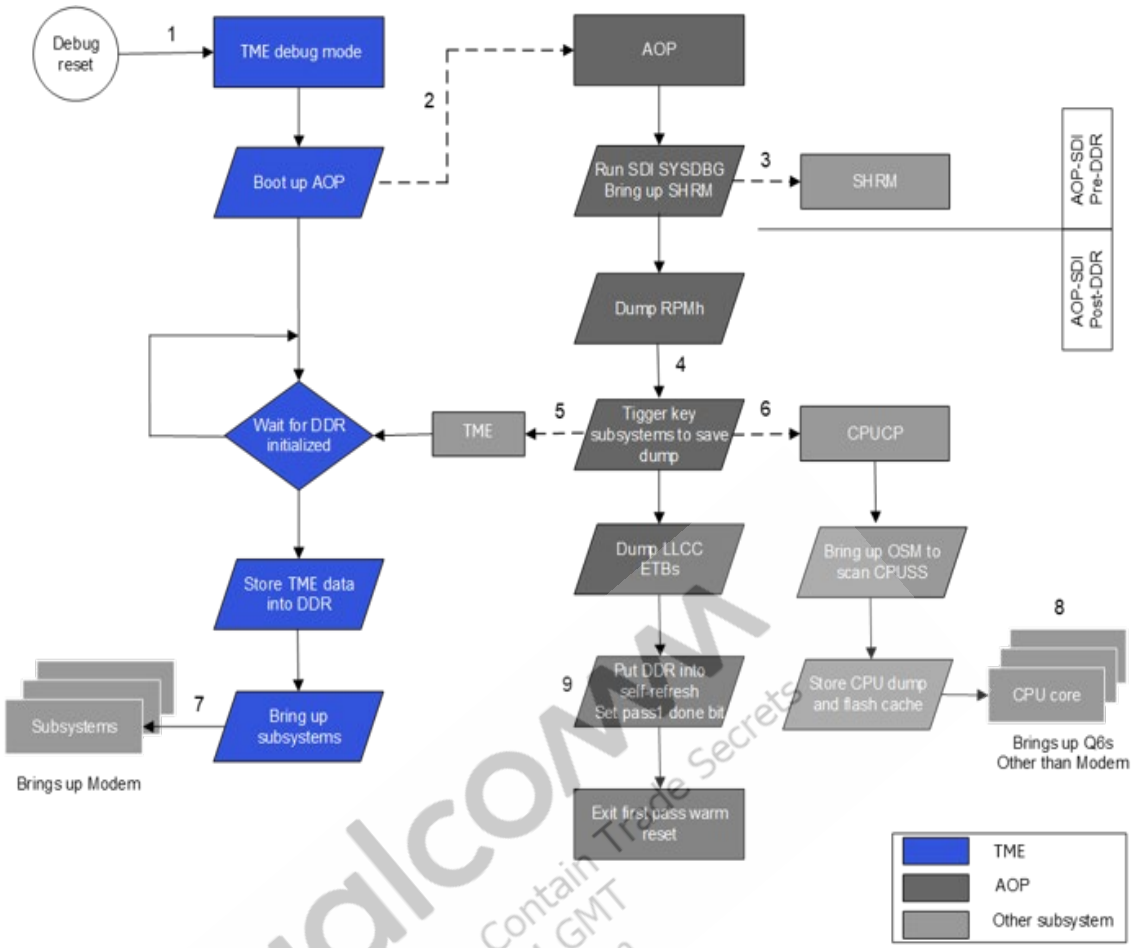
5. Second pass:
   - The system collects RAM dumps through Download mode.



4.9.2 Reset debug flow (first pass)

1. If Ramdump is enabled by debug policy, the first pass runs from TME.
2. TME prepares debug environment then brings AOP to run AOP-SDI code in OCIMEM.
3. AOP sets up the clocks necessary to bring up SHRM, issues SDI recover RSI command to DDR-AUX, SHRM brings DDR out of self-refresh. AOP sets a timer to poll DDR exit self-refresh completion, if timeout then goes to second pass.
4. Once DDR exits self-refresh, AOP saves RPMh registers, then triggers key subsystems (CPUCP and TME) to dump their corresponding data.
5. TME detects DDR ready and runs out of WFI state.
6. CPUCP brings up OSM to collect CPUSS scan dump, brings up CPU cores to dump and flash caches.
7. TME brings up Modem to collect dumps.
8. TZ SDI on boot core brings up QDSP6 subsystems other than Modem to collect dumps.
9. After AOP triggers key subsystems, it continues to dump local ETBs (SSC, LPASS, and so on). AOP also dumps system cache (LLCC) if needed, and puts DDR back to self-refresh, then exits the first Pass and triggers warm reset.

Qualcomm Confidential – May contain Trade Secrets 2026-02-27 05:37:34 GMT hyungchul.won@lge.com

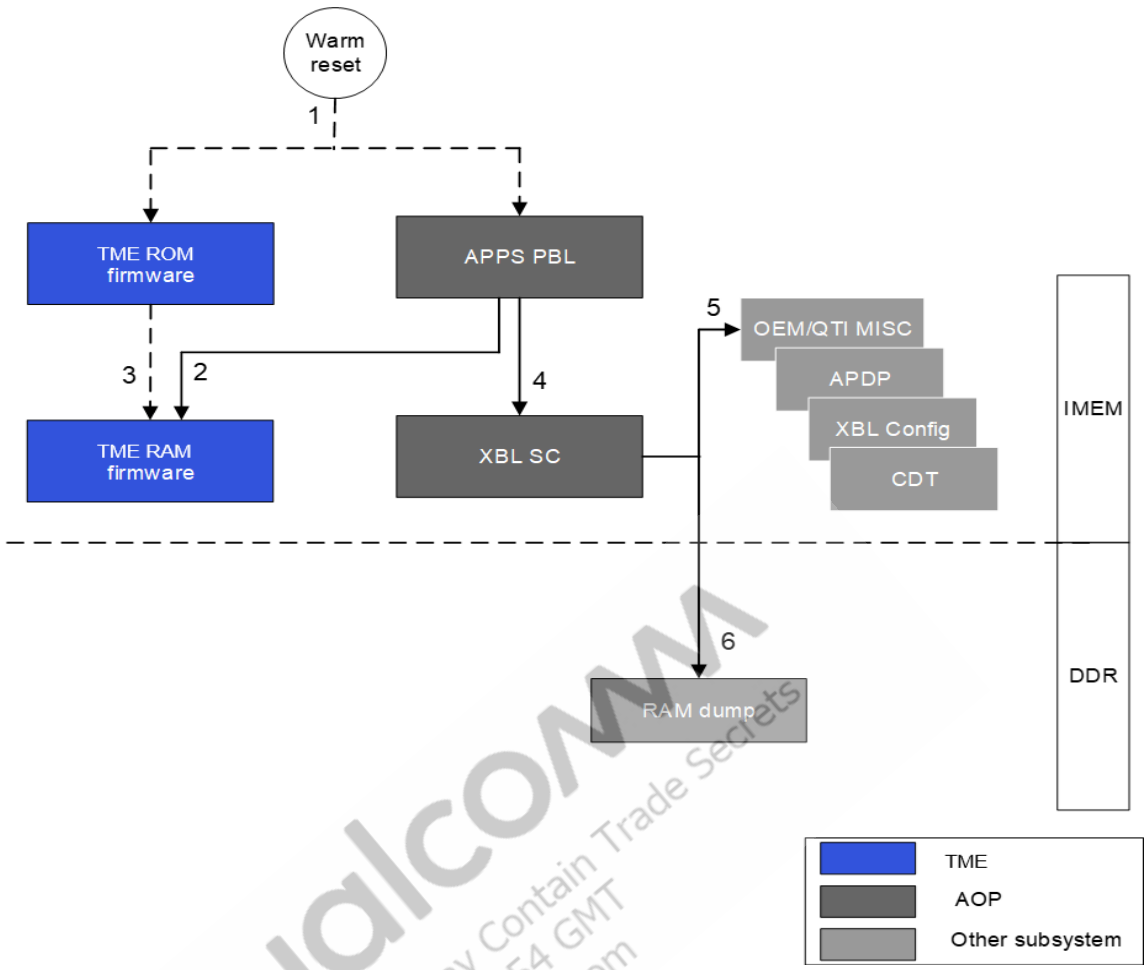


Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyunchul.woo@lge.com

4.9.3 Reset Debug Flow (Second Pass)

1. SDI exits first pass with warm reset, brings up both TME ROM firmware and APPS PBL.
2. APPS PBL loads TME firmware into TME IMEM.
3. TME ROM firmware authenticates TME firmware and brings it out of reset.
4. APPS PBL loads XBL SC to boot IMEM and TME ROM firmware authenticates XBL SC. After TME is ready and XBL SC image authentication is passed, TME permits APPS PBL exit to run XBL SC.
5. XBL SC loads MISC, APDP, XBL CONFIG, CDT. TME authenticates the images, if necessary. XBL SC detects it is in the Download mode and skips loading unnecessary images.

   XBL SC loads RAM dump images and TME authenticates the image. XBL SC detects it's in the Download mode, then jumps into RAM dump to collect RAM dump by Sahara or saves to SD card. XBL SC and TME should guarantee to sanitize security data according to RAM dump.



Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyungchul.won@lge.com

5 Unified Extensible Firmware Interface(UEFI)

UEFI development includes two main components: the application and the driver. The EFI loader is responsible for loading and running both types of EFI loadable images. All executable files—whether drivers or applications—use the .efi file extension.

For the UEFI specification, see https://github.com/tianocore/tianocore.github.io/wiki/EDK-II/.

5.1 UEFI driver

UEFI drivers operate by consuming and producing protocols—software interfaces that reside in memory and provide services to other components. These protocols form the UEFI's modular architecture.

UEFI drivers fall into two main categories:

- DXE drivers: These are loaded during the Driver Execution Environment (DXE) phase. Most system initialization occurs here using EFI boot services.
- UEFI drivers: These implement module-specific functionality and are typically loaded after DXE.

For UEFI drivers, it's considered best practice to define protocols according to the UEFI specification. If a protocol isn't standardized, the driver should be implemented as a DXE driver instead.

Each UEFI driver must declare an entry point function, which is invoked when the driver is loaded. For example, in the I²C UEFI driver:

- `MODULE_TYPE` is set to `DXE_DRIVER`.
- The entry point function is `I2CProtocolInit`.

Qualcomm Confidential – May Contain Trade Secrets
2026-02-27 05:27:54 GMT
hyungchul.yoon@lge.com

```
I2CDxe.inf
BASE_NAME                = I2C
MODULE_TYPE              = DXE_DRIVER
VERSION_STRING           = 1.0
  ENTRY_POINT             = I2CProtocolInit
[LibraryClasses]
  I2CLib
```

5.2 UEFI application

A UEFI application consumes protocols but doesn't produce them. Once the application exits, it's removed from memory. For UEFI applications, the `MODULE_TYPE` is set to `UEFI_APPLICATION`.

```
QcomChargerApp.inf
MODULE_TYPE                    = UEFI_APPLICATION
ENTRY_POINT                    = QcomChargerApp_Entry
```

A UEFI application provides the following capabilities:

- Platform and factory diagnostics to identify, isolate, and troubleshoot hardware issues.
- Configuration utilities for system setup and customization.
- Driver prototyping, allowing validation of driver functionality as an application before converting it into a UEFI driver.
- Use-case specific applications tailored to platform-defined requirements.

While the application emphasizes user interaction, the driver is responsible for delivering boot and runtime services. Protocols are configured by the user during application execution.

5.3 UEFI library

The UEFI library is designed to link platform or device-specific functionalities (APIs) to one or more UEFI drivers and UEFI applications.

The UEFI driver links the library using `LIBRARY_CLASS`.For example, the `LIBRARY_CLASS` name is `I2CLib`.

```
I2CLib.inf
VERSION_STRING                 = 1.0
LIBRARY_CLASS                  = I2CLib
# I2C Library
  I2CLib|QcomPkg/Library/I2CLib/I2CLib.inf
```

For more information about UEFI development, see https://github.com/tianocore/tianocore.github.io/wiki/UEFI-EDKII-Learning-Dev.

Build a custom protocol

Implement dynamic linking using a protocol. Use libraries for static linking. UEFI drivers access protocol interfaces produced by other modules using protocol services.

Access a UEFI protocol, which is a structure of function pointers associated with a global unique identifier (GUID), using a handle. Each executable image in UEFI has a handle protocol in the handle database.

The UEFI driver adds the UEFI protocol to the system table using the gBS->InstallProtocol(...) interface. Different modules retrieve the protocol interface using LocateProtocol().

The UEFI specification defines a set of boot services to handle protocols, including:

- Services to install protocols:
  - InstallProtocolInterface()
  - ReInstallProtocolInterface()
  - InstallMultipleProtocolInterfaces()
- Services to retrieve protocols:
  - LocateProtocol()
  - HandleProtocol()

Qualcomm Confidential - May Contain Trade Secrets 2026-03-27 05:37:55 GMT hwangchanwon@lge.com

- OpenProtocol()

The following example demonstrates how to use `RamPartitionProtocol`, available in the `QcomPkg/Include/Protocol/EFIRamPartition.h` file. This file contains all the declarations and implementations done in the EnvDXE driver. The UEFI application uses the protocol to print the RAM partitions.

Include the GUID definition associated with this protocol as follows:

```
/*  Protocol GUID definition */
/** @ingroup efi_ramPartition_protocol */
#define EFI_RAMPARTITION_PROTOCOL_GUID \
{ 0x5172FFB5, 0x4253, 0x7D51, { 0xC6, 0x41, 0xA7, 0x01, 0xF9, 0x73, 0x10,
0x3C } }
/** @cond */
/**
External reference to the RAMPARTITION Protocol GUID defined in the .dec
file.
*/
extern EFI_GUID gEfiRamPartitionProtocolGuid;
/** @endcond */
```

The following code define the functions and structures for this protocol:

```
typedef struct _RamPartition {
UINT64 Base;
UINT64 AvailableLength;
}RamPartitionEntry; typedef
EFI_STATUS
(EFIAPI *EFI_RAMPARTITION_GETRAMPARTITIONVERSION)( IN
EFI_RAMPARTITION_PROTOCOL *This,
OUT UINT32  *MajorVersion,
OUT UINT32  *MinorVersion );
typedef EFI_STATUS
(EFIAPI *EFI_RAMPARTITION_GETHIGHESTBANKBIT)
```

The following `EnvDxe` driver code shows the RAM partition table function:

```
STATIC EFI_RAMPARTITION_PROTOCOL RamPartitionProtocol =
{
  EFI_RAMPARTITION_PROTOCOL_REVISION,
  EFI_GetRamPartitionVersion,
  EFI_GetHighestBankBit,
  EFI_GetRamPartitions,
  EFI_GetMinPasrSize,
  EFI_GetInstalledPhysicalMemory,
  EFI_GetPreLoadedRamPartitions
};


EFI_STATUS
SetupRamPartitionProtocol ()
{
  EFI_STATUS Status;
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyung-chul.won@lge.com

```
  STATIC EFI_HANDLE ImageHandle;

  Status = InitRamPartitionTableLib();
  if (Status != EFI_SUCCESS)
    return EFI_UNSUPPORTED;

  Status = gBS->InstallMultipleProtocolInterfaces (&ImageHandle,

&gEfiRamPartitionProtocolGuid,
                                                  &RamPartitionProtocol,
                                                  NULL, NULL);
  return Status;
}
```

Implement UEFI DXE driver

The DXE drivers are responsible for initializing the processor, chip products, and platform components. Additionally, they provide software abstractions for system services, console devices,

and boot devices.

For more information about UEFI DXE, see https://uefi.org/specs/PI/1.8/V2_Overview.html.

Before you implement the UEFI DXE driver:

- Create the `<uefidriver_name>.inf` file for the driver at `boot_images/boot/QcomPkg/Drivers/<uefidriver_name>/<uefidriver_name>.inf`.
- Implement the functions produced by this UEFI driver.
- Install the protocols to consume the functions implemented by this UEFI driver.

To implement the UEFI DXE driver, follow these steps:

- Add the driver in the description (DSC) file of the package and the flash description file (FDF). Go to the `boot_images\boot\QcomPkg\Drivers` build directory to add a UEFI driver that implements the UEFI protocols.
  This example shows the implementation of RamPartitionProtocol declared in Build a custom protocol in the `QcomPkg/Drivers/EnvDxe/EnvDxe.c` UEFI driver.
- Create the EnvDxe.inf file for the driver at QcomPkg/Drivers/EnvDxe/EnvDxe.inf. Ensure that the MODULE_TYPE entry is DXE_RUNTIME_DRIVER or DXE_DRIVER.

The following are the contents of the `EnvDXE.inf` file:

```
[Defines]
INF_VERSION = 0x00010005
BASE_NAME   = EnvDxe
FILE_GUID   = 90A49AFD-422F-08AE-9611-E788D3804845
MODULE_TYPE = DXE_RUNTIME_DRIVER
VERSION_STRING  = 1.0
ENTRY_POINT = EnvDxeInitialize
[Sources]
EnvDxe.c
[BuildOptions.AARCH64]
```

```
#GCC:*_*_*_CC_FLAGS = -O0
#MSFT:*_*_*_CC_FLAGS = -O0
[Packages] MdePkg/MdePkg.dec ArmPkg/ArmPkg.dec
MdeModulePkg/MdeModulePkg.dec QcomPkg/QcomPkg.dec
[LibraryClasses] UefiDriverEntryPoint UefiBootServicesTableLib
UefiRuntimeServicesTableLib QcomBaseLib
DebugLib ProcLib PcdLib FBPTLib
PeCoffGetEntryPointLib BaseLib
UefiLib DxeServicesTableLib RamPartitionTableLib QcomLib
UefiCfgLib SerialPortLib
[GUID]
gEfiEventFBPTOsLoaderLoadImageStartGuid
gEfiEventFBPTOsLoaderStartImageStartGuid
gEfiEventFBPTExitBootServicesEntryGuid
gEfiEventFBPTExitBootServicesExitGuid gEfiEventVirtualAddressChangeGuid
gEfiDebugImageInfoTableGuid gQcomFwVersionHexGuid
gBlockIoRefreshGuid
[Protocols] gEfiCpuArchProtocolGuid gEfiTimerArchProtocolGuid
gEfiRamPartitionProtocolGuid
[FixedPCD] gQcomTokenSpaceGuid.PcdFBPTPayloadBaseOffset
gArmTokenSpaceGuid.PcdCpuVectorBaseAddress
[Depex] TRUE
```

4. Implement the functions declared in `QcomPkg/Include/Protocol/EFIRamPartition.h`.

```
EFI_STATUS
EFI_GetRamPartitionVersion (
IN EFI_RAMPARTITION_PROTOCOL *This,
OUT UINT32  *MajorVersion,
OUT UINT32  *MinorVersion
){
if ((MajorVersion == NULL) || (MinorVersion == NULL) ) return
EFI_INVALID_PARAMETER;
*MajorVersion = 2;
*MinorVersion = 0; return EFI_SUCCESS;}
EFI_STATUS
EFI_GetHighestBankBit (
IN EFI_RAMPARTITION_PROTOCOL *This,
OUT UINT32  *HighestBankBit){
if (HighestBankBit == NULL) return EFI_INVALID_PARAMETER;
*HighestBankBit = 0;
#ifdef PRE_SIL
return EFI_UNSUPPORTED;
#else
return EFI_SUCCESS;
#endif } EFI_STATUS
EFI_GetRamPartitions (
IN EFI_RAMPARTITION_PROTOCOL *This,
OUT RamPartitionEntry   *RamPartitions, IN OUT UINT32   *NumPartition
){
```

Qualcomm Confidential - May Contain Trade Secrets 2026-02-27 05:37:54 GMT hyung-chul.woo@lge.com

```
EFI_STATUS  Status;
MemRegionInfo *pRamPartTable = NULL; UINTN  AvailNumParts = 0;
UINTN   i   = 0;
if (NumPartition == NULL) return EFI_INVALID_PARAMETER;
Status = GetRamPartitions(&AvailNumParts, NULL); if (Status ==
EFI_BUFFER_TOO_SMALL){
if (AvailNumParts > (UINTN)(*NumPartition)){
```

5. Install the protocol using the `InstallMultipleProtocolinterfaces` function.

```
STATIC EFI_RAMPARTITION_PROTOCOL RamPartitionProtocol = {
EFI_RAMPARTITION_PROTOCOL_REVISION,
EFI_GetRamPartitionVersion, EFI_GetHighestBankBit, EFI_GetRamPartitions
};
EFI_STATUS
SetupRamPartitionProtocol ()
{EFI_STATUS Status;
STATIC EFI_HANDLE ImageHandle;
Status = InitRamPartitionTableLib(); if (Status != EFI_SUCCESS)
return EFI_UNSUPPORTED;
Status = gBS->InstallMultipleProtocolInterfaces (&ImageHandle,
&gEfiRamPartitionProtocolGuid,
return Status;
&RamPartitionProtocol, NULL, NULL); }
```

6. Add the driver in the description file of the package.

In the following example, add `EnvDxe.inf` to the `<chipset>Pkg_Core.dsc` file.

```
QcomPkg/Drivers/EnvDxe/EnvDxe.inf {
<LibraryClasses> DebugLib|QcomPkg/Library/DebugLib/DebugLib.inf }
```

7. Add the driver to the flash description file. For example, add `EnvDxe.inf` to the `<chipset>Pkg.fdf` file.

Build and add a UEFI application

1. Use the UEFI test application in QcomPkg/QcomTestCommon/EnvTest as an example.

   Create the EnvTest.inf file for the application.
   The INF file has information about the application as shown in the following sample.
   For the UEFI application, the MODULE_TYPE entry should be UEFI_APPLICATION. The following is a sample EnvTest.inf file:

```
[Defines]
INF_VERSION = 0x00010005
BASE_NAME   = EnvTest
     FILE_GUID   = 8CEC0EDF-EF17-425E-8F6A-4B46274FB7C9 MODULE_TYPE
= UEFI_APPLICATION
VERSION_STRING  = 1.0
ENTRY_POINT = EnvTestMain
#
```

```
# The following information is for reference only and not required by the
build tools.
#
#   VALID_ARCHITECTURES = ARM AARCH64
#
[Sources]
EnvTest.c
[BuildOptions.AARCH64]
#GCC:*_*_*_CC_FLAGS = -O0
[Packages] MdePkg/MdePkg.dec
MdeModulePkg/MdeModulePkg.dec QcomPkg/QcomPkg.dec
[Protocols] gEfiRamPartitionProtocolGuid
[LibraryClasses] UefiApplicationEntryPoint UefiLib
TestInterfaceLib MemoryAllocationLib
```

The following table lists the main section definitions of the INF file.

Table 5-1 File definitions (UEFI driver and application INF files)

Item	Description
`INF_VERSION`	Identifies the INF version.
`BASE_NAME`	The name of the driver or application.
`FILE_GUID`	The unique GUID of the driver or application.
`MODULE_TYPE`	The type of module for the driver or application. For example, UEFI_APPLICATION
`VERSION_STRING`	The version number of the string to be used, typically 1.0.
`ENTRY_POINT`	The entry function of the driver or application.
`[Sources]`	The names of the source files in the driver or, application, listed line by line.
`[Packages]`	The different module packages used in the driver or, application, listed line by line.
`[LibraryClasses]`	The libraries that are linked into the code are listed line by line.
`[Guids]`	The list of the global GUID names that are used by the driver or application.
`[Protocols]`	The UEFI protocols consumed or produced by the driver or application.
`[FeaturePcd]( if necessary)`	The feature allows runtime selection of code from the platform configuration database (PCD).
`[Pcd] (if necessary)`	The fixed PCD values used in this component.
`[Depex] (if necessary)`	A list of dependent drivers that must be loaded before this driver or application can load or run.

In the UEFI application, the source file is at `QcomPkg/ QcomTestPkg/EnvTest/EnvTest.c`. The following is a simplified explanation of the roles:

- `#include<Uefi.h>`: Required header file for every UEFI application. It includes the types and constructs defined in UEFI.
- `#include<Library/UefiApplicationEntryPoint.h>`: This library contains the functions required to define the entry point of the UEFI application.
- `#include<Library/UefiBootServicesTableLib.h>`: This library allows the UEFI application to access boot services.

- `#include<Library/UefiRuntimeServicesTableLib.h>`: This library allows the UEFI application to access runtime services.

2. Define the entry function for the application.

The application must have an entry function defined in the source code that indicates where the application begins to run. For example, list the prototype of the entry function in `EnvTest.c`.

For the UEFI application entry point, there are two parameters:

- ImageHandle: This is the image handle of the UEFI application.
- SystemTable: This is the pointer to the EFI system table.

```
/**
Entry point for the application
@param[in] ImageHandle  Image handle
@param[in] SystemTable  Pointer to the System Table
@retval EFI_SUCCESS Execution successful
@retval other   Error occurred
\**/ EFI_STATUS EFIAPI
EnvTestMain (
IN EFI_HANDLE   ImageHandle, IN EFI_SYSTEM_TABLE    *SystemTable
){
EFI_STATUS  Status; TEST_START("EnvTest");
Status = RamPartProtocolTest(); TestStatus("EnvTest", Status);
TEST_STOP("EnvTest");
return EFI_SUCCESS;}
```

The entry function calls RamPartProtocolTest to enable the application to display the RAM partitions.

```
STATIC EFI_STATUS
RamPartProtocolTest (VOID){
EFI_STATUS  Status = EFI_NOT_FOUND;
EFI_RAMPARTITION_PROTOCOL   *pRamPartProtocol = NULL; RamPartitionEntry
*pRamPartitions = NULL;
UINT32  NumPartitions = 0;
UINT32  i = 0;
Status = gBS->LocateProtocol(&gEfiRamPartitionProtocolGuid, NULL,
(VOID**)&pRamPartProtocol);
if (EFI_ERROR(Status) || (&pRamPartProtocol == NULL)){
DEBUG((EFI_D_ERROR, "Locate EFI_RAMPARTITION_Protocol failed, Status
=   (0x%x)\r\n", Status)); return EFI_NOT_FOUND;}
Status = pRamPartProtocol->GetRamPartitions (pRamPartProtocol, NULL,
&NumPartitions);
if (Status == EFI_BUFFER_TOO_SMALL){
pRamPartitions = AllocatePool (NumPartitions * sizeof (RamPartitionEntry));
if (pRamPartitions == NULL) return EFI_OUT_OF_RESOURCES;
Status = pRamPartProtocol->GetRamPartitions (pRamPartProtocol,
pRamPartitions, &NumPartitions);
if (EFI_ERROR (Status) || (NumPartitions < 1) )      {
DEBUG((EFI_D_ERROR, "Failed to get RAM partitions")); return EFI_NOT_FOUND;
}}
```

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:37:54 GMT mungchul.wongoge.com

```
DEBUG ((EFI_D_WARN, "\n------------------\n"));
DEBUG ((EFI_D_WARN, "+ RAM Partitions +\n"));
DEBUG ((EFI_D_WARN, "------------------\n"));
DEBUG((EFI_D_WARN, "Base \t\t\t Available Length\n")); for (i = 0; i <
NumPartitions; i++) {
DEBUG((EFI_D_WARN, "0x%016lx \t 0x%016lx\n", pRamPartitions[i].Base,
pRamPartitions[i].AvailableLength));}
return EFI_SUCCESS;}
```

3. Use `LocateProtocol()` to obtain the `gEfiRamPartitionProtocolGuid` protocol interface.
4. Use `GetRamPartitions` to obtain partition information once `LocateProtocol()` provides the interface information.

Add application to description files

To build the application with the relevant package, add it to the packager's description (DSC) file.

Additionally, to include the application in the flash image, add it to the flash description (FDF) file of the corresponding package.

Add the application in the description (DSC) file of the package

Add the INF file of the application to the DSC file.

Since the Qualcomm UEFI test application is part of QcomTestPkg, add it to the package-specific DSC file.

For example, add `QcomTestPkg` to `QcomTestPkg.dsc`, at `QcomPkg/QcomTestPkg/QcomTestPkg.dsc`.

In this case, add `EnvTest.inf` to the `QcomTestPkg.dsc` file.

```
## Test Applications
#INF QcomPkg/QcomTestPkg/EnvTest/EnvTest.inf
```

Use devictree in UEFI

The XBL DeviceTree feature supports `pre-ddr-<chipset>-1.0.dtb`, `post-ddr-<chipset>-1.0.dtb`, and `platform.dtb` DeviceTree blobs (DTB).

- The `pre-ddr-<chipset>-1.0.dtb` and `post-ddr-<chipset>-1.0.dtb` contains the configurations that are loaded before and after DDR is initialized.
- The `platform.dtb` contains the devicetree configuration.

Configure devicetree

UEFI uses a devicetree format packaged into the XBL configuration to support predefined customizations.

The following steps outline the process to generate a DeviceTree binary:

1. Qualcomm distributes DeviceTree source (DTS) and DTSI files within the boot software.
2. Add your own devicetree configurations for UEFI.
3. Build the boot image. For more information, see Build BOOT.
4. The new devicetree configuration is available as part of `xbl_config.elf`. Flash the `xbl_config.elf` image.

Qualcomm Confidential – May Contain Trade Secrets 2026-02-27 05:37:54 GMT wongsul.won@tme.com

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
XBLConfig binary item. See XBL Configuration Guide	80-PE663-1
QDUTT eCDT User Guide	80-NR058-9
Non-HLOS Customization Using Device Tree Hardware/Software Settings User Guide	80-29623-65
SW6100 Technical Reference Manual and Chipset Introduction	80-83950-5
SW6100 PMIC Software User Guide	80-74841-80
SW6100 GPIO Overview	80-74841-92
SW6100 Peripheral (UART, SPI, I2C, I3C, USB) Overview	80-74841-87

A.2 Acronyms and terms

Acronym or term	Definition
aDSP	Applications digital signal processor
AOP	Always on processor
cDSP	Compute digital signal processor
CDT	Configuration data table
NSP	Neural signal processing
TZ	TrustZone
UEFI	Unified extensible firmware interface

Qualcomm
Confidential – May Contain Trade Secrets
2026-2-27 05:53:54 GMT
bwengchu@lge.com

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Qualcomm Confidential - May Contain Trade Secrets
2026-02-27 05:37:54 GMT
by yuechul.won@lge.com